PORTFOLIO PROJECT 01

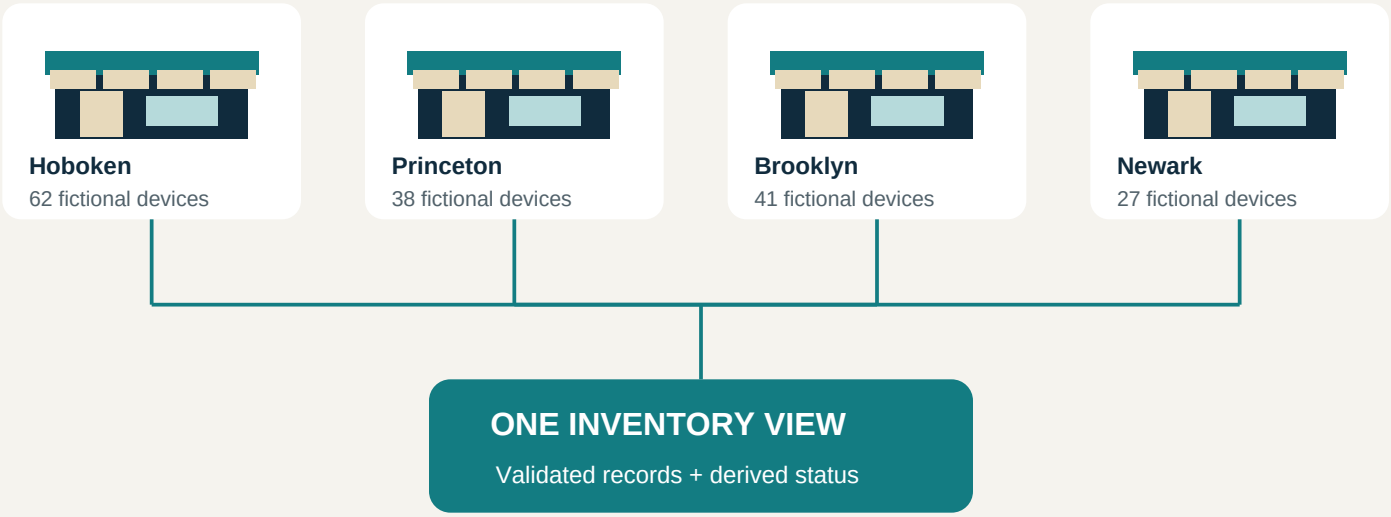
Harborline

Retail IT Operations

Inventory visibility across the store network

A fictional apparel retailer provides the setting for an inventory application that brings device identity, connection recency, support lifecycle, and active faults into one consistent view.

FOUR STORES. ONE EXPLAINABLE DATASET.



Built by Vic, drawing on enterprise IT experience and retail operations knowledge. Designed to demonstrate practical Python development, reliable validation, and API design.

Fictional brand and records. Fixed-time demo; no live telemetry.

Know what needs attention

A register that has stopped checking in and a printer reporting a jam need different explanations. Harborline preserves those distinctions so inventory can support an analyst's next action.

1
IMPORT
Structured inventory

→

2
VALIDATE
Check the whole file

→

3
REVIEW
Explain each status

Three signals, one health result

CONNECTION

Online, Stale, Offline, or Unknown based on device-specific check-in windows.

SUPPORT

Supported, nearing end, unsupported, unknown, or not applicable.

FAULTS AND VERSION LAG

Critical and warning issues, plus catalog release lag for applicable platforms.

ILLUSTRATIVE RULE EXAMPLE

Receipt printer

Connection: Stale Issue: critical paper jam

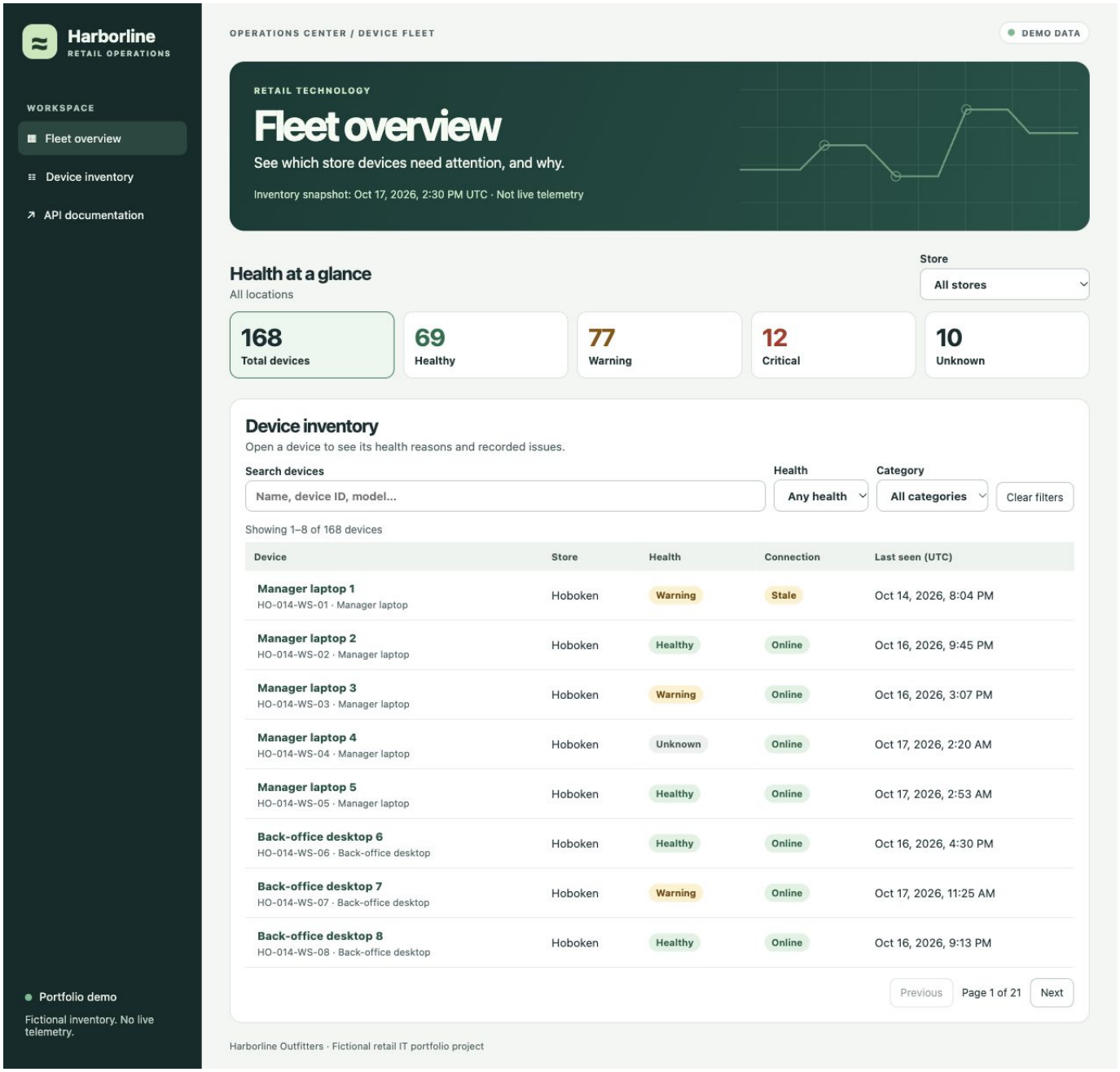
Result: Critical health, with both reasons retained.

Why it matters: a device can be recently observed and still have a serious fault. Offline devices are also Critical, so connection and health totals are intentionally separate.

In the working dashboard, select a store, filter by health or category, search the inventory, and expand a device to see why it needs attention.

From store to device

Start with fleet health, narrow the inventory to one store, then open a device to see its support status, recorded issues, and suggested action.



Fleet overview in light mode, captured from the locally running Harborline demo. All inventory and device statuses are fictional.

Try the workflow: Select Hoboken, choose Critical, and expand a device to review the reasons behind its status.

Business context. Reliable behavior.

The portfolio value comes from making operational rules explicit, reproducible, and inspectable.

168
DEVICE RECORDS

4
STORE LOCATIONS

25
DEVICE TYPES USED

JSON INVENTORY
Stores / catalog / devices

→

PYTHON RULES
Validate / derive / explain

→

FASTAPI
Dashboard / device API

Deterministic sample data

Fixed seed and evaluation time produce a repeatable demonstration. Store counts are calculated from the generated records.

Whole-inventory validation

Device identity, allowed types, catalog references, timestamps, and cross-record relationships are checked before data is exposed.

Read-only API with clear failures

The reviewed API returns device records with calculated statuses and demo metadata; invalid inventory is intended to return 503 without partial device data.

IMPLEMENTATION EVIDENCE

Independently reproduced: 168 records; store totals 62 / 38 / 41 / 27; deterministic generation. Latest reviewed inventory: 69 Healthy, 77 Warning, 12 Critical, 10 Unknown. Subsequent developer-reported results: 60 validator checks and 19 API tests passing; that revision was not independently rerun.

Portfolio edition 02 | November 2026
Harborline is the first project module in an expandable portfolio series. Future completed projects can follow this structure: operating problem, illustrated workflow, product value, and implementation evidence.

Scope: portfolio prototype. No production deployment, live monitoring, or measured savings are asserted.